JSON Validation and Completion Report USER INTENT: The user wants to convert a large text file containing educational module slides for "INFORMATIKA FUN" (specifically

https://gemini.google.com/app/4ec018f29ebc68a4

User prompt: Apakah ini yang dimaksud VITE_FIREBASE_DATABASE_URL?

Response: Berdasarkan gambar tangkapan layar yang kamu berikan, halaman tersebut menampilkan dasbor "Realtime Database" di dalam project Firebase Console milikmu. Di bagian tengah, tepat di samping ikon tautan (rantai), terdapat sebuah URL yaitu `[https://databasemateriinformatika-default-rtdb.asia-southeast1.firebasedatabase.app](https://databasemateriinformatika-default-rtdb.asia-southeast1.firebasedatabase.app)`. URL ini merupakan alamat (*endpoint*) utama yang disediakan oleh Firebase agar aplikasi luar bisa berkomunikasi dengan databasemu. Dengan demikian, URL tersebut adalah nilai yang benar untuk diisikan ke dalam variabel `VITE_FIREBASE_DATABASE_URL`.

Cara Menerapkan URL ke dalam Kode

Salin tautan tersebut secara langsung dari baris yang memiliki ikon rantai, dan pastikan kamu tidak menyertakan karakter tambahan seperti spasi atau tulisan `null` yang ada di area pratinjau data di bawahnya.

Jika kamu menempatkannya di dalam file konfigurasi seperti `.env` pada project Vite, tuliskan dengan format berikut:

```
VITE_FIREBASE_DATABASE_URL="https://databasemateriinformatika-default-rtdb.asia-southeast1.firebasedatabase
```

Penjelasan Singkat

- **Fungsi Variabel:** Variabel ini akan dipanggil oleh kodemu saat melakukan inisialisasi Firebase, sehingga aplikasimu tahu persis server database mana yang harus dihubungi untuk membaca atau menyimpan data.
- **Keamanan:** Menaruhnya di dalam variabel `.env` dengan awalan `VITE_` memastikan bahwa Vite dapat memproses URL ini dengan aman saat kamu melakukan proses *build* atau *deploy* aplikasimu nanti.

Apakah ada bagian konfigurasi Firebase lainnya di kodemu yang perlu kita periksa atau hubungkan bersama-sama hari ini?

User prompt: Jika website yang saya buat memuat banyak data dari hasil pengerjaan siswa di firebase, apakah ini berpengaruh pada performa website saya? Jika saya menggunakan database firestore dan database realtime dengan jangka waktu yang lama secara gratis apakah akan ada resiko kehilanan data? Berikan saya solusi terbaik agar saya bisa menggunakan website saya dengan pengguna ratusan orang tapi tetap dengan performa yang baik

Response: Memuat banyak data sekaligus dari Firebase tentu akan berdampak pada performa website. Waktu muat (*loading*) bisa menjadi lambat, memori peramban (*browser*) pengguna akan terkuras, dan yang paling penting, kuota gratis harian Firebase kamu bisa habis dengan sangat cepat.

Terkait kekhawatiranmu soal paket gratis (Spark Plan): **Tidak ada risiko data hilang atau dihapus oleh Google** meski digunakan dalam jangka waktu lama. Namun, ada batas kuota ketat (misalnya, Firestore hanya mengizinkan 50.000 kali baca/hari, dan Realtime Database membatasi 1 GB penyimpanan serta 10 GB unduhan/bulan). Jika ratusan pengguna memuat data terus-menerus dan kuota ini terlampaui, database akan menolak permintaan baru. Efeknya, website tidak bisa menampilkan atau menyimpan data sampai batas waktu direset (esok harinya), tetapi data yang sudah ada tetap aman.

Agar website tetap cepat dan aman dari batas kuota untuk ratusan pengguna, kita harus mengubah cara kode meminta data. Berikut adalah solusi dan langkah-langkah penerapannya:

1. Membatasi Data dengan Pagination (Limit & Cursor)

Jangan pernah mengambil seluruh data siswa sekaligus. Ambil data sebagian, misalnya 10 siswa pertama. Saat pengguna menekan tombol "Muat Lebih Banyak", barulah kita ambil 10 data berikutnya.

Contoh Penerapan di Firestore:

```
import { getFirestore, collection, query, orderBy, limit, startAfter, getDocs } from "firebase/firestore";

const db = getFirestore(app); // Asumsi 'app' adalah inisialisasi Firebase-mu
const siswaRef = collection(db, "data_siswa");

// Langkah A: Mengambil 10 data pertama
let dokumenTerakhir = null; // Variabel untuk mengingat posisi terakhir data

async function muatDataPertama() {
  const q = query(siswaRef, orderBy("nama"), limit(10));
  const snapshot = await getDocs(q);

  // Simpan posisi dokumen terakhir untuk digunakan nanti
  dokumenTerakhir = snapshot.docs[snapshot.docs.length - 1];

  snapshot.forEach((doc) => {
    console.log(doc.id, " => ", doc.data());
    // Tampilkan data ini ke HTML/UI website
```

```
  });
}

// Langkah B: Mengambil 10 data selanjutnya (saat tombol ditekan)
async function muatDataSelanjutnya() {
  if (!dokumenTerakhir) return; // Hentikan jika tidak ada data sebelumnya

  const q = query(siswaRef, orderBy("nama"), startAfter(dokumenTerakhir), limit(10));
  const snapshot = await getDocs(q);

  dokumenTerakhir = snapshot.docs[snapshot.docs.length - 1]; // Perbarui posisi terakhir

  snapshot.forEach((doc) => {
    console.log(doc.id, " => ", doc.data());
  });
}
```

2. Mengaktifkan Cache Lokal (Offline Persistence)

Ini adalah fitur luar biasa dari Firestore. Saat diaktifkan, Firestore akan menyimpan salinan data yang pernah dimuat ke dalam memori perangkat pengguna. Jika pengguna memuat ulang halaman, Firestore tidak akan memotong kuota harianmu dari server, melainkan langsung menampilkan data dari *cache* lokal.

Contoh Penerapan di Firestore:

```
import { initializeApp } from "firebase/app";
import { getFirestore, enableIndexedDbPersistence } from "firebase/firestore";

// Konfigurasi API Key kamu yang kita bahas sebelumnya
const firebaseConfig = { /* ... */ };
const app = initializeApp(firebaseConfig);
const db = getFirestore(app);

// Mengaktifkan penyimpanan cache lokal
enableIndexedDbPersistence(db)
  .catch((err) => {
    if (err.code == 'failed-precondition') {
      console.log("Beberapa tab terbuka sekaligus, cache gagal diaktifkan di tab ini.");
    } else if (err.code == 'unimplemented') {
      console.log("Browser ini tidak mendukung fitur cache lokal.");
    }
  });
```

3. Meratakan Struktur Data (Khusus Realtime Database)

Jika kamu lebih banyak menggunakan **Realtime Database**, aturan utamanya adalah: **jangan membuat struktur data bersarang (nested) yang terlalu dalam.** Saat kamu meminta sebuah data dari Realtime Database, Firebase akan mengunduh *seluruh* data anak di bawahnya.

Jangan lakukan ini:

```
{
  "kelas_10": {
    "siswa_1": {
      "nama": "Budi",
      "tugas": { "tugas1": "A", "tugas2": "B", "tugas3": "A" /* ratusan riwayat tugas */ }
    }
  }
}
```

Jika kamu hanya ingin menampilkan daftar nama di halaman depan, mengambil `kelas_10` akan memuat seluruh riwayat tugas secara tidak perlu, menghabiskan memori dan kuota data.

Lakukan ini (Data Flattening): Pisahkan profil siswa dan riwayat tugasnya ke dalam "koleksi" (*node*) yang berbeda.

```
{
  "profil_siswa": {
    "siswa_1": { "nama": "Budi", "kelas": "10" }
  },
  "tugas_siswa": {
    "siswa_1": { "tugas1": "A", "tugas2": "B" }
  }
}
```

Dengan struktur ini, saat kamu menampilkan daftar siswa, kamu cukup memuat `profil_siswa` yang datanya sangat kecil dan ringan.

Apakah dari ketiga solusi di atas ada yang ingin kita terapkan langsung ke dalam kodemu saat ini? Kita bisa mulai dari merancang *pagination* jika kamu mau.

User prompt: Jika pada menu penerapan konsep konten yang sudah saya buat bisa dikirim ke clasroom bagaimana caranya?

Response: Untuk mengirim konten dari websitemu ke Google Classroom, kita memiliki dua pilihan metode. Sebagai *partner coding*-mu, aku akan menjelaskan keduanya agar kamu bisa memilih mana yang paling pas dengan alur kerjamu.

1. Menggunakan Tombol "Share to Classroom" (Sederhana & Cepat) Cara ini sangat mudah. Kita hanya perlu membuat sebuah tombol dengan tautan khusus. Ketika diklik, pengguna akan melihat jendela *pop-up* resmi dari Google, di mana mereka bisa memilih sendiri kelasnya dan membagikan tautan dari website milikmu (bisa sebagai pengumuman atau tugas).

2. Menggunakan Google Classroom API (Tingkat Lanjut) Cara ini digunakan jika kamu ingin websitemu yang bekerja di balik layar untuk otomatis membuatkan "Tugas" (Assignment) atau "Materi" (Course Work) langsung ke dalam kelas pengguna. Ini membutuhkan penyiapan otentikasi login dengan akun Google (OAuth 2.0) di Google Cloud Console.

Mari kita mulai dengan membuat **Opsi 1 (Tombol Share)** terlebih dahulu karena cara ini sangat ringan dan bisa langsung kamu tempelkan pada project yang sedang kita kerjakan.

Kode Penerapan Tombol Share Classroom

Kita hanya membutuhkan sedikit kode HTML dan JavaScript untuk membuat pop-up bagikan ini berfungsi.

HTML: Letakkan tombol ini di dalam halaman "Penerapan Konsep" tempat kontenmu berada.

```
<!-- Tombol untuk memicu pop-up Google Classroom -->
<button id="btnShareClassroom">Kirim ke Google Classroom</button>
```

JavaScript: Salin dan tempelkan kode ini ke dalam file JavaScript milikmu.

```
// 1. Kita ambil elemen tombol dari HTML menggunakan ID-nya
const btnShare = document.getElementById("btnShareClassroom");

// 2. Kita beri perintah agar sistem mendengarkan klik pada tombol tersebut
btnShare.addEventListener("click", () => {

  // Variabel ini berisi URL halaman web atau konten yang ingin dibagikan.
  // Pastikan URL ini sudah online (bukan localhost) agar Classroom bisa membacanya.
  const urlKontenKamu = "https://website-kamu.com/halaman-konten-spesifik";

  // Ini adalah format URL wajib dari Google untuk fitur "Share"
  // encodeURIComponent memastikan karakter spesial pada URL-mu aman untuk dikirim
  const urlClassroom = `https://classroom.google.com/share?url=${encodeURIComponent(urlKontenKamu)}`;

  // Membuka jendela pop-up baru berukuran kecil agar terlihat rapi
  window.open(
    urlClassroom,
    "ShareToClassroom",
    "width=600,height=600,scrollbars=yes"
  );
});
```

Penjelasan Parameter yang Bisa Disesuaikan:

- `urlKontenKamu`: Ini adalah variabel paling penting. Jika setiap siswa mengerjakan konten yang berbeda dan menghasilkan URL unik (misalnya `[website.com/hasil?id=123](https://website.com/hasil?id=123)`), kamu harus membuat JavaScript mengambil URL dinamis tersebut sebelum memasukkannya ke dalam tombol.

Apakah metode pop-up bagikan ini sudah sesuai dengan yang kamu bayangkan, atau kamu membutuhkan integrasi yang lebih otomatis menggunakan Google Classroom API?

User prompt: Jika saya terlanjur sudah membuat konten pada maisng-maisn menu solusi atau opsi mana yang paling cocok agar konten soal tersebut dapat terhubung ootomatis ke clasroom

Response: Halo! Wah, hebat sekali kamu sudah selesai membuat konten-konten soalnya di masing-masing menu. Karena kamu menginginkan agar konten tersebut bisa terhubung secara **otomatis** ke Google Classroom (tanpa pengguna harus mengatur pop-up secara manual), maka solusi yang paling tepat adalah **Opsi 2: Menggunakan Google Classroom API**.

Dengan metode ini, websitemu akan terhubung langsung ke sistem Google. Misalnya, saat seorang guru menekan tombol "Terbitkan" di websitemu, kodemu akan otomatis membuatkan materi atau tugas di kelas yang mereka pilih.

Berikut adalah penjelasan dan panduan untuk menerapkannya.

Penjelasan Umum Solusi

Kita akan menggunakan pustaka resmi dari Google (Google API Client Library) di dalam kodemu. Prosesnya terdiri dari dua tahap utama:

1. **Otentikasi (Login):** Meminta izin kepada pengguna (guru) agar websitemu boleh mengakses Google Classroom mereka.
2. **Kirim Data (API Call):** Mengirimkan data konten soalmu (judul, deskripsi, link) ke Classroom mereka sebagai "Tugas" (CourseWork).

Langkah 1: Persiapan di Google Cloud (Wajib)

Sebelum menulis kode, websitemu butuh "kunci akses" dari Google.

1. Kunjungi Google Cloud Console.
2. Buat project baru.
3. Cari menu **APIs & Services** > **Library**, lalu cari dan aktifkan **Google Classroom API**.
4. Buka menu **Credentials**, klik **Create Credentials** > **OAuth client ID**.
5. Pilih tipe aplikasi "Web application", masukkan URL websitemu, lalu kamu akan mendapatkan **Client ID** (bentuknya seperti `12345...apps.googleusercontent.com`). Simpan ID ini.

Langkah 2: Penerapan Kode

Berikut adalah kode dasar untuk memicu login Google dan mengirimkan konten soalmu menjadi tugas di Classroom.

1. Tambahkan Library Google ke HTML-mu: Tambahkan kedua baris ini di dalam tag `<head>` atau tepat sebelum tag penutup `</body>` di file HTML-mu.

```
<!-- Library untuk memanggil API Google -->
<script async defer src="https://apis.google.com/js/api.js"></script>
<!-- Library untuk memunculkan tombol Login Google Identity -->
<script async defer src="https://accounts.google.com/gsi/client"></script>
```

2. Kode JavaScript untuk Otomatisasi: Salin kode ini ke file JavaScript-mu. Baca komentar di dalam kode untuk memahami setiap langkahnya.

```
// Ganti dengan Client ID yang kamu dapatkan dari Google Cloud Console
const CLIENT_ID = 'KETIK_CLIENT_ID_KAMU_DISINI.apps.googleusercontent.com';

// Scope adalah izin yang kita minta (izin membaca kelas & membuat tugas)
const SCOPES = 'https://www.googleapis.com/auth/classroom.courses.readonly https://www.googleapis.com/auth/

let tokenClient;
let accessToken = null;

// Fungsi ini akan dipanggil untuk menyiapkan tombol login Google
function inisialisasiGoogleAPI() {
  tokenClient = google.accounts.oauth2.initTokenClient({
    client_id: CLIENT_ID,
    scope: SCOPES,
    callback: (response) => {
      if (response.error) {
        console.error("Gagal login:", response.error);
        return;
      }
      accessToken = response.access_token;
      console.log("Berhasil terhubung! Siap mengirim data.");

      // Setelah login berhasil, panggil fungsi kirim konten
      kirimKontenKeClassroom();
    },
  });
}

// Fungsi untuk memicu login
function mulaiKoneksiClassroom() {
  if (accessToken === null) {
    // Memunculkan jendela pop-up login Google
    tokenClient.requestAccessToken({ prompt: 'consent' });
  } else {
    // Jika sudah login, langsung kirim
    kirimKontenKeClassroom();
  }
}

// Fungsi utama untuk mengirim soal otomatis ke Classroom
async function kirimKontenKeClassroom() {
  // Asumsi ID kelas sudah diketahui (kamu bisa membuat fungsi terpisah untuk mengambil daftar kelas)
  const idKelas = "KODE_ID_KELAS_GURU";

  // Data soal/konten dari menu websitemu yang akan dikirim
  const dataTugas = {
    title: "Tugas Interaktif: Penerapan Konsep", // Judul tugas
    description: "Silakan kerjakan soal dari website berikut ini.",
    workType: "ASSIGNMENT", // Jenis konten: Tugas
    state: "PUBLISHED", // Langsung diterbitkan
    materials: [
      {
        link: {
```

```
          // URL halaman soalmu yang spesifik
          url: "https://website-kamu.com/soal-penerapan-konsep"
        }
      }
    ]
  };

  try {
    // Memanggil API Classroom untuk membuat tugas
    const response = await fetch(`https://classroom.googleapis.com/v1/courses/${idKelas}/courseWork`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${accessToken}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(dataTugas)
    });

    const hasil = await response.json();
    console.log("Tugas berhasil dibuat di Classroom!", hasil);
    alert("Konten berhasil dikirim ke Classroom secara otomatis!");

  } catch (error) {
    console.error("Gagal mengirim tugas:", error);
  }
}

// Menjalankan inisialisasi saat halaman web selesai dimuat
window.onload = function() {
  inisialisasiGoogleAPI();
};
```

Penjelasan Variabel yang Bisa Disesuaikan

- `CLIENT_ID`: Harus diganti dengan kunci dari Google Cloud milikmu.
- `idKelas`: Ini adalah ID dari kelas Google Classroom guru. Untuk tahap selanjutnya, kita bisa membuat kode agar guru bisa memilih kelas mereka dari menu *dropdown* yang kita ambil otomatis dari API.
- `dataTugas`: Objek ini berisi kerangka kontenmu (Judul, deskripsi, dan link materi).

Pertanyaan untuk langkah selanjutnya: Apakah kita perlu membuatkan kode tambahan agar websitemu bisa menarik "Daftar Kelas" milik guru tersebut terlebih dahulu, sehingga mereka bisa memilih ke kelas mana konten soal ini akan dikirim secara otomatis?

User prompt: SAYA INGIN MEMPERBARUI HALAMAN LOGIN UNTUK SISWA: KOTAK LOGINNYA MEMUAT EMAIL DAN PASSWORD SESUAI DENGAN KOTAK LOGIN HALAMAN GURU, APA YANG SAYA HARUS LAKUKAN DLU, APAKAH SAYA MENGHAPUS DATA DIFIREBASE ATAU SAYA BISA MEMPERBARUI ATAU MENGEDIT DIHALAMAN FIREBASE?

Response: Halo! Senang bisa kembali membantumu coding. Membuat halaman login siswa sama persis dengan halaman guru (menggunakan Email dan Password) adalah langkah yang sangat bagus agar sistem keamanan di websitemu lebih rapi dan konsisten.

Menjawab pertanyaan utamamu: **Kamu TIDAK perlu menghapus data apapun di Firebase.** Data siswa yang sudah kamu miliki saat ini tetap aman. Kita bisa memperbarui dan menyesuaikannya secara langsung.

Berikut adalah penjelasan umum tentang solusi kita dan apa yang harus dilakukan pertama kali.

Penjelasan Umum Solusi

Untuk mengubah login siswa menjadi Email dan Password, prosesnya terbagi menjadi dua bagian:

1. **Di sisi Firebase (Server):** Kita hanya perlu memastikan fitur login Email/Password sudah aktif, lalu mendaftarkan email dan password untuk masing-masing siswa di menu *Authentication*.
2. **Di sisi Kode (Website):** Kita akan memperbarui tampilan (HTML) login siswa agar memiliki dua kotak input (Email dan Password), lalu mengubah kode JavaScript-nya agar memanggil fungsi login Firebase yang benar.

Langkah-Langkah Penerapan

Langkah 1: Apa yang harus dilakukan di Firebase Console? Kamu tidak perlu mengubah kodingan terlebih dahulu, mulailah dari dasbor Firebase:

1. Buka project Firebase kamu dan masuk ke menu **Authentication**.
2. Pilih tab **Sign-in method**. Pastikan status penyedia **Email/Password** sudah dalam keadaan *Enabled* (Aktif). Karena gurumu sudah pakai ini, kemungkinan besar ini sudah aktif.
3. Masuk ke tab **Users** di menu Authentication. Di sinilah kamu bisa mengklik tombol **Add user** untuk membuatkan akun (Email dan Password) secara manual untuk siswa, atau membiarkan siswa mendaftar sendiri nanti jika kamu membuatkan halaman registrasi.

Langkah 2: Memperbarui Tampilan Kode HTML Di file HTML halaman login siswa, ubah kotak inputnya agar menyerupai milik guru. Kira-kira bentuknya akan seperti ini:

```
<!-- Kotak Login Siswa -->
<div id="login-siswa">
  <h2>Login Siswa</h2>

  <label for="emailSiswa">Email:</label>
  <input type="email" id="emailSiswa" placeholder="Masukkan email siswa" required>

  <label for="passwordSiswa">Password:</label>
  <input type="password" id="passwordSiswa" placeholder="Masukkan password" required>

  <button id="btnLoginSiswa">Masuk</button>
</div>
```

Langkah 3: Memperbarui Kode JavaScript Di file JavaScript-mu, kita akan menggunakan fungsi `signInWithEmailAndPassword` dari Firebase. Berikut adalah contoh kodenya:

```
// Mengimpor fungsi otentikasi dari Firebase
import { getAuth, signInWithEmailAndPassword } from "firebase/auth";
import { app } from "./firebase-config.js"; // Sesuaikan dengan file konfigurasimu

const auth = getAuth(app);

// Mengambil elemen dari HTML
const btnLoginSiswa = document.getElementById("btnLoginSiswa");
const inputEmail = document.getElementById("emailSiswa");
const inputPassword = document.getElementById("passwordSiswa");

// Memberikan perintah saat tombol ditekan
btnLoginSiswa.addEventListener("click", () => {
  const email = inputEmail.value;
  const password = inputPassword.value;

  // Memanggil Firebase untuk memeriksa kecocokan email dan password
  signInWithEmailAndPassword(auth, email, password)
    .then((userCredential) => {
      // Jika berhasil login
      const user = userCredential.user;
      console.log("Berhasil login sebagai siswa:", user.email);
      alert("Login berhasil! Mengalihkan ke halaman siswa...");

      // Kode untuk berpindah halaman, misalnya:
      // window.location.href = "/halaman-siswa.html";
    })
    .catch((error) => {
      // Jika gagal (email salah atau password salah)
      console.error("Gagal login:", error.message);
      alert("Login gagal. Periksa kembali email dan password-mu.");
    });
});
```

Meminta Klarifikasi (Pertanyaan untukmu)

Agar aku bisa memberikan panduan kode selanjutnya yang 100% cocok dengan project-mu, bolehkah kamu memberitahuku: **Sebelumnya, siswa menggunakan cara apa untuk login ke dalam website?** Apakah menggunakan nama saja, NIS (Nomor Induk Siswa), kode kelas, atau yang lainnya?

Beri tahu aku agar kita bisa menghubungkan data lama mereka dengan metode login Email/Password yang baru ini!

User prompt: JI SAYA SUDAH MELAKUKAN LANGKAH TERSEBUT KEMUDIAN BAGAIMANA DENGAN SISWA PADA FIRESTORE APAKAH INI TIDAK MENGGANGU TAU DOUBLE

Response: Halo! Pertanyaan yang sangat bagus. Kamu sangat teliti karena memikirkan integrasi antara sistem *login* dan *database*.

Kabar baiknya: **Tidak, ini tidak akan mengganggu atau membuat data siswamu menjadi ganda (double).**

Berikut adalah penjelasan umum tentang alasannya dan bagaimana kita akan menyambungkan keduanya agar bekerja dengan sempurna.

Penjelasan Umum

Di Firebase, **Authentication** (tempat kamu mengatur email/password) dan **Firestore** (tempat kamu menyimpan data siswa seperti pada gambar) adalah dua sistem yang berdiri sendiri. Membuat akun baru di menu *Authentication* tidak akan otomatis membuat dokumen baru di koleksi `siswa` milikmu, sehingga tidak akan ada data ganda.

Namun, karena keduanya berdiri sendiri, kita perlu "**menghubungkan**" akun login (Authentication) dengan data profil mereka di Firestore.

Melihat gambar yang kamu lampirkan, dokumen di dalam koleksi `siswa` saat ini menggunakan ID acak (seperti `00NRVeNDmExfDDdNK33Z`) dan berisi data seperti `Kelas`, `NISN`, `Nama_Siswa`, `No`, dan `Status_Akun`. Saat ini, belum ada data yang bisa mencocokkan dokumen tersebut dengan akun login (email).

Langkah-langkah Solusi (Cara Menghubungkannya)

Agar saat siswa *login* sistem tahu persis data mana yang merupakan milik mereka, kita perlu melakukan dua hal:

Langkah 1: Tambahkan Data Email ke Firestore (Manual di Console) Karena kamu akan menggunakan email untuk login, kita harus mendaftarkan email tersebut ke dalam dokumen siswa.

1. Buka halaman Firestore seperti pada gambar yang kamu lampirkan.
2. Pilih salah satu dokumen siswa (misalnya milik "NUR WAFIQAH").
3. Klik tombol **+ Add field**.
4. Masukkan **Field**: `Email` (tipe string), dan **Value**: masukkan alamat email yang sama persis dengan yang kamu daftarkan untuk siswa tersebut di menu *Authentication*.
5. Ulangi langkah ini untuk siswa lainnya.

Langkah 2: Memperbarui Kode Login JavaScript Setelah login berhasil menggunakan Email/Password, kita akan menyuruh kode untuk mencari data siswa di Firestore berdasarkan email yang baru saja dipakai login.

Berikut adalah kode yang sudah diperbarui:

```
// Mengimpor fungsi auth dan firestore
import { getAuth, signInWithEmailAndPassword } from "firebase/auth";
import { getFirestore, collection, query, where, getDocs } from "firebase/firestore";
import { app } from "./firebase-config.js";

const auth = getAuth(app);
const db = getFirestore(app); // Menginisialisasi Firestore

const btnLoginSiswa = document.getElementById("btnLoginSiswa");
const inputEmail = document.getElementById("emailSiswa");
const inputPassword = document.getElementById("passwordSiswa");

btnLoginSiswa.addEventListener("click", () => {
  const email = inputEmail.value;
  const password = inputPassword.value;

  // 1. Proses Login Authentication
  signInWithEmailAndPassword(auth, email, password)
    .then(async (userCredential) => {
      const user = userCredential.user;
      console.log("Berhasil login Auth:", user.email);

      // 2. Proses Mencari Data di Firestore berdasarkan Email
      // Kita mencari di koleksi "siswa" di mana field "Email" sama dengan email yang login
      const q = query(collection(db, "siswa"), where("Email", "==", user.email));
      const querySnapshot = await getDocs(q);

      if (!querySnapshot.empty) {
        // Jika data ditemukan di Firestore
        querySnapshot.forEach((doc) => {
          const dataSiswa = doc.data();
          console.log("Data Firestore ditemukan untuk:", dataSiswa.Nama_Siswa);

          // Opsional: Simpan nama atau NISN ke sessionStorage agar bisa dipakai di halaman lain
          sessionStorage.setItem("namaSiswa", dataSiswa.Nama_Siswa);
          sessionStorage.setItem("nisnSiswa", dataSiswa.NISN);
          sessionStorage.setItem("kelasSiswa", dataSiswa.Kelas);

          alert(`Selamat datang, ${dataSiswa.Nama_Siswa}!`);
          // Arahkan ke halaman beranda siswa
          // window.location.href = "/beranda-siswa.html";
        });
      } else {
        // Jika siswa bisa login, tapi kamu belum menambahkan field 'Email' di Firestore-nya
        alert("Login berhasil, tapi profil data di Firestore belum ditemukan. Hubungi Guru.");
      }
    })
    .catch((error) => {
      console.error("Gagal login:", error.message);
      alert("Login gagal. Periksa kembali email dan password-mu.");
    });
});
```

Penjelasan Singkat Kode di Atas:

1. `signInWithEmailAndPassword`: Memeriksa keabsahan email dan password.
2. `query(..., where("Email", "==", user.email))`: Ini adalah fungsi pencarian. Setelah Firebase memvalidasi login, ia akan pergi ke koleksi `siswa` dan mencari dokumen mana yang memiliki `Email` yang cocok.
3. `sessionStorage`: Ini fitur bawaan peramban (browser) untuk mengingat data (seperti Nama atau NISN) sementara waktu, sehingga di halaman selanjutnya (misalnya halaman soal) kamu bisa memanggil nama siswa tanpa harus memuat ulang dari database.

Apakah kamu ingin mulai menambahkan *field* `Email` tersebut ke beberapa siswa di Firestore sekarang untuk kita jadikan bahan percobaan (*testing*) kodenya?

User prompt: Untuk langkah 1 apakah ada cara otomatis untuk menggunggah data email tersebut, karena ada ratusan siswa yang akan diinput

Response: Halo! Pertanyaan yang sangat cerdas dan efisien. Tentu saja, memasukkan ratusan data secara manual satu per satu sangat melelahkan dan rentan terjadi kesalahan ketik. Sebagai programmer, kita selalu punya cara untuk mengotomatiskannya!

Untuk membantumu dengan tepat, **aku perlu memahami sedikit lebih detail (Pahami permintaanku):** Saat ini, apakah kamu memiliki daftar email siswa tersebut di dalam file Excel (Spreadsheet) / CSV? Atau apakah email siswa dibuat menggunakan pola tertentu (misalnya gabungan NISN: `136633563@sekolah.com`)?

Sambil menunggu jawabanmu, aku akan memberikan panduan standar yang paling sering digunakan untuk menyelesaikan masalah ini.

Penjelasan Umum tentang Solusinya

Solusi terbaiknya adalah membuat sebuah **Skrip Update Massal (Batch Update Script)** menggunakan JavaScript di websitemu. Cara kerjanya:

1. Kita akan menyiapkan data siswa (misal: NISN dan pasangannya yaitu Email) dalam bentuk format kode JavaScript (disebut *Array of Objects*). Data ini bisa kamu salin-tempel (copy-paste) dari Excel.
2. Saat kamu menekan sebuah tombol khusus (tombol Admin), skrip akan berjalan menyusuri daftar tersebut satu per satu.
3. Skrip akan mencari data siswa di database Firestore berdasarkan `NISN`.
4. Jika ketemu, skrip akan otomatis menambahkan kolom `Email` ke dalam dokumen siswa tersebut menggunakan fungsi `updateDoc`.

Dengan cara ini, ratusan data bisa diperbarui hanya dalam beberapa detik!

Kode dan Petunjuk Penerapannya

Berikut adalah contoh skrip JavaScript yang bisa kamu tempelkan di halaman khusus (misalnya halaman admin yang hanya bisa diakses olehmu).

1. Siapkan HTML-nya (Halaman Admin) Buat satu tombol sederhana untuk memicu proses pembaruan massal ini:

```
<div style="padding: 20px; text-align: center;">
  <h2>Panel Admin: Update Email Massal</h2>
  <button id="btnUpdateMassal" style="padding: 10px 20px; background: blue; color: white;">Mulai Update Ra
  <p id="statusUpdate">Status: Menunggu...</p>
</div>
```

2. Siapkan Kode JavaScript-nya Pastikan kamu mengimpor fungsi `updateDoc` dari Firebase. Berikut kodenya:

```
import { getFirestore, collection, query, where, getDocs, updateDoc, doc } from "firebase/firestore";
import { app } from "./firebase-config.js";

const db = getFirestore(app);
const btnUpdate = document.getElementById("btnUpdateMassal");
const textStatus = document.getElementById("statusUpdate");

// Ini adalah data yang bisa kamu salin-tempel dari file Excel/CSV milikmu.
// Kamu bisa mengubahnya ke format JSON/Array seperti ini secara online dengan tool "Excel to JSON converte
const dataEmailSiswa = [
  { nisn: "136633563", email: "nur.wafiqah@sekolah.com" },
  { nisn: "987654321", email: "siswa.kedua@sekolah.com" },
  { nisn: "112233445", email: "siswa.ketiga@sekolah.com" }
  // ... Tambahkan ratusan data lainnya di sini
];

// Fungsi untuk melakukan pembaruan massal
async function prosesUpdateMassal() {
  textStatus.innerText = "Status: Memulai proses update...";
  btnUpdate.disabled = true; // Matikan tombol agar tidak terklik 2 kali

  let sukses = 0;
  let gagal = 0;

  // Melakukan perulangan untuk setiap siswa di dalam daftar
  for (let i = 0; i < dataEmailSiswa.length; i++) {
    const data = dataEmailSiswa[i];
```

```
    try {
      // 1. Cari dokumen siswa berdasarkan NISN (karena NISN itu unik)
      const q = query(collection(db, "siswa"), where("NISN", "==", parseInt(data.nisn)));
      // Catatan: Gunakan parseInt() jika NISN di Firestore bertipe angka (int64) seperti di gambarmu sebel

      const querySnapshot = await getDocs(q);

      if (!querySnapshot.empty) {
        // 2. Jika ketemu, ambil ID dokumen acak tersebut dan lakukan Update
        querySnapshot.forEach(async (dokumenSiswa) => {
          const refDokumen = doc(db, "siswa", dokumenSiswa.id);

          // Memasukkan field 'Email' ke dalam dokumen
          await updateDoc(refDokumen, {
            Email: data.email
          });
        });
        sukses++;
        console.log(`Berhasil update email untuk NISN: ${data.nisn}`);
      } else {
        gagal++;
        console.log(`Gagal: Siswa dengan NISN ${data.nisn} tidak ditemukan.`);
      }
    } catch (error) {
      gagal++;
      console.error(`Error pada NISN ${data.nisn}:`, error);
    }
  }

  textStatus.innerText = `Status: Selesai! Berhasil: ${sukses}, Gagal: ${gagal}`;
  alert("Proses update massal selesai!");
}

// Pasang perintah klik pada tombol
btnUpdate.addEventListener("click", prosesUpdateMassal);
```

Cara Menggunakan Kode Ini:

1. Letakkan kode ini di halaman admin.
2. Siapkan data siswa dari Excel, lalu ubah ke format seperti variabel `dataEmailSiswa` di atas. Jika kamu bingung cara mengubah Excel ke bentuk kode itu, beri tahu aku, nanti aku ajari trik mudahnya!
3. Buka halaman tersebut di *browser*, klik tombol "**Mulai Update Ratusan Email**".
4. Buka dasbor Firebase Firestore-mu, dan kamu akan melihat kolom `Email` sudah terisi secara otomatis untuk semua siswa yang terdaftar di kodemu!

Apakah bentuk data asalmu adalah Excel? Beri tahu aku agar kita bisa menyesuaikan kodenya dengan lebih sempurna!

User prompt: Saya mempunyai daftar email siswa yaitu akun belajar id

Response: Halo! Bagus sekali, menggunakan akun Belajar.id adalah pilihan yang sangat tepat dan standar yang baik untuk sistem informasi sekolah.

1. Memahami Permintaanku Agar kita bisa memasukkan ratusan akun Belajar.id ini ke dalam kode pembaruan massal (*batch update*) yang kita bahas sebelumnya, aku perlu mengonfirmasi format datamu:

- Apakah daftar email tersebut saat ini tersimpan di dalam file **Excel** atau **Google Sheets**?
- Apakah di dalam daftar file tersebut sudah terdapat **NISN** yang bersandingan dengan **Email** masing-masing siswa? (Kita membutuhkan NISN ini sebagai kata kunci agar skrip tidak salah sasaran saat menyimpan email di Firestore).

2. Penjelasan Umum Solusi Jika data ratusan siswamu sudah ada di Excel (misalnya, Kolom A untuk NISN dan Kolom B untuk Email), kamu sama sekali tidak perlu mengetik ulang menjadi kode satu per satu.

Kita akan menggunakan trik sederhana menggunakan rumus (*formula*) Excel. Rumus ini akan secara otomatis menggabungkan teks dari kolom NISN dan Email menjadi format kode JavaScript (*Array of Objects*). Setelah itu, kamu cukup melakukan *Copy* (Salin) dari Excel dan *Paste* (Tempel) langsung ke dalam file kodemu.

3. Petunjuk Penerapan (Mengubah Data Excel menjadi Kode)

Asumsikan data di Excel atau Google Sheets milikmu seperti ini:

- **Sel A2:** Berisi data NISN (contoh: `136633563`)
- **Sel B2:** Berisi Email (contoh: `nur.wafiqah@smp.belajar.id`)

Lakukan langkah berikut:

1. Klik pada **Sel C2** (kolom kosong di sebelah email).
2. Salin dan tempel rumus berikut ini ke dalam Sel C2, lalu tekan Enter: `="{ nisn: """ & A2 & """, email: """ & B2 & """ },"`
3. Hasil di Sel C2 akan otomatis berubah menjadi bentuk kode seperti ini: `{ nisn: "136633563", email: "nur.wafiqah@smp.belajar.id" },`
4. Tarik ujung kotak Sel C2 ke bawah untuk menerapkan rumus tersebut ke ratusan siswa lainnya di bawahnya.
5. Jika sudah, salin (Copy) seluruh hasil yang ada di kolom C tersebut.
6. Buka file kode JavaScript Admin yang kita buat sebelumnya, lalu tempelkan (Paste) di dalam kotak `[` dan `]` pada variabel `dataEmailSiswa`.

Kira-kira nanti kodenya akan terlihat rapi seperti ini:

```
// Data ini didapatkan dari hasil copy-paste rumus Excel
const dataEmailSiswa = [
  { nisn: "136633563", email: "nur.wafiqah@smp.belajar.id" },
  { nisn: "987654321", email: "siswa.dua@smp.belajar.id" },
  // ... ratusan data lainnya akan berjejer di sini otomatis
];
```

Bagaimana menurutmu? Apakah daftar datamu di Excel sudah siap untuk kita olah dengan rumus ini?

User prompt: PROMPT ANALISIS FILE DAN PENYUSUNAN SOAL AKM BERDASARKAN TUJUAN PEMBELAJARAN Bertindaklah sebagai ahli penyusun soal Asesmen Kompetensi Minimum (AKM), pengembang asesmen pembelajaran Informatika SMP Fase D, dan editor dokumen pembelajaran yang teliti dan profesional. Saya melampirkan sebuah file yang memuat data tujuan pembelajaran, topik pembelajaran, serta kolom soal atau soal games yang perlu dilengkapi. Tugas Anda adalah menganalisis seluruh isi file terlampir dengan saksama, kemudian menyusun soal AKM untuk setiap pertemuan atau tujuan pembelajaran sesuai dengan topik pembelajaran yang tercantum dalam file tersebut. A. ANALISIS FILE TERLAMPIR Sebelum menyusun soal, lakukan analisis secara menyeluruh terhadap file terlampir. 1. Baca dan periksa seluruh isi file, mulai dari pertemuan pertama hingga pertemuan terakhir. 2. Identifikasi setiap pertemuan, tujuan pembelajaran, topik pembelajaran, serta kolom yang harus diisi. 3. Pastikan setiap tujuan pembelajaran dipahami secara tepat sebelum membuat soal. 4. Perhatikan hubungan antara tujuan pembelajaran, materi, dan kompetensi yang harus dikuasai siswa. 5. Jangan membuat soal secara umum atau sekadar mengambil soal dari materi lain yang tidak sesuai dengan tujuan pembelajaran. 6. Jangan melewatkan satu pun pertemuan atau tujuan pembelajaran yang terdapat dalam file. 7. Jika terdapat tujuan pembelajaran yang berbeda, buatlah soal yang berbeda dan benar-benar sesuai dengan tujuan tersebut. B. JUMLAH DAN JENIS SOAL Pada SETIAP PERTEMUAN ATAU TUJUAN PEMBELAJARAN, buatkan soal AKM dengan ketentuan berikut: Jenis SoalJumlahPilihan Ganda (PG)5 soalPilihan Ganda Kompleks (PGK)5 soalMenjodohkan5 soalIsian5 soalTotal20 soal Ketentuan ini berlaku untuk seluruh pertemuan tanpa terkecuali. Jika file memuat banyak pertemuan, pastikan seluruh pertemuan mendapatkan jumlah soal yang sama sesuai ketentuan. C. KARAKTERISTIK SOAL AKM Buatlah soal dengan karakteristik berikut: 1. Soal sesuai dengan tujuan pembelajaran dan topik pembelajaran pada setiap pertemuan. 2. Menggunakan bahasa Indonesia yang baik, jelas, komunikatif, dan mudah dipahami oleh siswa SMP. 3. Memiliki tingkat kesulitan menengah hingga tinggi yang sesuai dengan kemampuan siswa SMP. 4. Mengutamakan kemampuan berpikir tingkat tinggi (HOTS), terutama C4 (menganalisis), C5 (mengevaluasi), dan C6 (mencipta) apabila sesuai dengan tujuan pembelajaran. 5. Mengukur kemampuan siswa dalam memahami informasi, menganalisis situasi, menerapkan konsep, mengevaluasi pilihan, dan menyelesaikan masalah. 6. Menggunakan konteks kehidupan sehari-hari yang dekat dengan pengalaman siswa SMP. 7. Jika memungkinkan, gunakan stimulus berupa teks, tabel, ilustrasi situasi, data, atau kasus sederhana yang relevan dengan materi. 8. Hindari soal yang hanya menguji hafalan definisi atau sekadar mengingat istilah. 9. Jangan membuat soal yang terlalu rumit dari segi bahasa sehingga siswa kesulitan memahami maksud pertanyaan. 10. Pastikan setiap soal memiliki jawaban yang benar dan dapat dipertanggungjawabkan berdasarkan materi pembelajaran. D. KETENTUAN PILIHAN JAWABAN Untuk soal Pilihan Ganda dan Pilihan Ganda Kompleks, buatlah pilihan jawaban yang berkualitas dan tidak mudah ditebak oleh siswa. Perhatikan ketentuan berikut: 1. Pilihan jawaban harus logis, masuk akal, dan berkaitan erat dengan pertanyaan. 2. Buatlah pengecoh (distraktor) yang berasal dari kemungkinan kesalahan pemahaman siswa terhadap materi. 3. Hindari pilihan jawaban yang terlalu jelas salah atau tidak masuk akal. 4. Jangan menjadikan jawaban benar sebagai satu-satunya pilihan yang memiliki kalimat paling panjang, paling lengkap, atau paling terperinci. 5. Seimbangkan panjang kalimat dan struktur bahasa antara jawaban benar dan jawaban pengecoh. 6. Jangan memberikan petunjuk jawaban melalui penggunaan kata, pola kalimat, istilah, atau susunan pilihan. 7. Hindari pola jawaban benar yang berulang, misalnya selalu A atau selalu pilihan dengan kalimat terpanjang. 8. Jangan menggunakan pilihan seperti "Semua jawaban benar" atau "Semua jawaban salah" kecuali benar-benar diperlukan dan sesuai dengan tujuan asesmen. 9. Pastikan setiap pilihan jawaban memiliki tingkat kemiripan yang wajar sehingga siswa harus memahami materi untuk menentukan jawaban. 10. Periksa kembali seluruh pilihan jawaban agar tidak terdapat petunjuk yang dapat dimanfaatkan siswa untuk menebak jawaban. Prinsip utama: siswa harus menjawab berdasarkan pemahaman, penalaran, dan analisis, bukan berdasarkan pola atau petunjuk yang terdapat dalam pilihan jawaban. E. KETENTUAN SETIAP JENIS SOAL 1. Pilihan Ganda (5 soal) - Setiap soal memiliki satu jawaban yang benar. - Gunakan pilihan A, B, C, dan D. - Buat soal berbasis pemahaman, penerapan, analisis, atau pemecahan masalah. - Buat pilihan jawaban yang logis dan seimbang. - Hindari soal yang jawabannya dapat ditemukan hanya dengan menebak pola kalimat. 2. Pilihan Ganda Kompleks (5 soal) - Setiap soal memiliki lebih dari satu jawaban yang benar. - Gunakan pilihan A, B, C, dan D. - Berikan instruksi yang jelas, misalnya: "Pilihlah semua jawaban yang benar!" - Variasikan jumlah jawaban benar, yaitu 2 atau 3 pilihan yang benar sesuai kebutuhan. - Jangan membuat pola jumlah jawaban benar yang mudah ditebak. - Pastikan setiap pilihan dapat dinilai secara independen berdasarkan kebenarannya. - Hindari pilihan yang saling tumpang tindih atau membuat siswa bingung karena ketidakjelasan makna. 3. Menjodohkan (5 soal) - Setiap soal menjodohkan memuat 5 pasangan pernyataan dan jawaban. - Sajikan dalam bentuk tabel yang rapi. - Kolom pertama berisi pernyataan, situasi, konsep, atau kasus. - Kolom kedua berisi pilihan jawaban yang harus dipasangkan. - Buat pilihan jawaban yang relevan dengan tujuan pembelajaran. - Jika memungkinkan, tambahkan pilihan pengecoh agar siswa tidak hanya menggunakan metode eliminasi sederhana. - Pastikan setiap pasangan memiliki hubungan yang tepat dan tidak ambigu. 4. Isian (5 soal) - Buat soal isian yang menuntut siswa memberikan jawaban secara mandiri. - Gunakan pertanyaan yang jelas dan tidak menimbulkan penafsiran ganda. - Utamakan soal berbasis analisis, penerapan konsep, dan pemecahan masalah. - Hindari pertanyaan yang hanya meminta siswa menghafal istilah sederhana. - Pastikan jawaban yang diharapkan dapat dinilai secara objektif. - Jika terdapat beberapa kemungkinan jawaban yang benar, tentukan alternatif jawaban yang dapat diterima. F. PENGISIAN SOAL PADA FILE TERLAMPIR Setelah seluruh soal selesai disusun, masukkan soal tersebut langsung ke dalam file terlampir. Ketentuan pengisian: 1. Isi kolom kosong yang telah disediakan untuk soal atau soal games. 2. Sesuaikan soal dengan struktur tabel, format, dan susunan dokumen yang sudah ada. 3. Jangan mengubah tujuan pembelajaran, topik pembelajaran, materi, atau isi lain yang sudah benar dalam file. 4. Jangan menghapus kolom, baris, tabel, maupun informasi yang sudah tersedia. 5. Jangan mengubah urutan pertemuan. 6. Jangan memindahkan soal ke pertemuan yang tidak sesuai. 7. Jangan menggabungkan beberapa tujuan pembelajaran menjadi satu kelompok soal apabila file telah memisahkannya. 8. Pertahankan format asli dokumen sejauh memungkinkan. 9. Jika kolom soal games memiliki format tertentu, ikuti format tersebut agar soal dapat digunakan

sesuai kebutuhan. 10. Pastikan seluruh soal tersusun secara rapi, konsisten, dan mudah dibaca. G. PEMERIKSAAN KUALITAS SOAL Sebelum menyelesaikan pekerjaan, lakukan pemeriksaan ulang secara menyeluruh. Periksa hal-hal berikut: - Seluruh pertemuan dalam file telah dianalisis. - Setiap tujuan pembelajaran memiliki 20 soal. - Setiap pertemuan memuat 5 PG, 5 PGK, 5 menjodohkan, dan 5 isian. - Seluruh soal sesuai dengan tujuan pembelajaran dan topik pembelajaran. - Soal memiliki tingkat kesulitan yang sesuai untuk siswa SMP. - Soal mengutamakan kemampuan berpikir tingkat tinggi. - Pilihan jawaban tidak memberikan petunjuk jawaban benar. - Panjang pilihan jawaban tidak menunjukkan pola jawaban benar. - Tidak terdapat soal yang ambigu atau memiliki lebih dari satu jawaban benar pada soal PG. - Jawaban pada soal PGK telah diperiksa satu per satu. - Pasangan soal menjodohkan telah diperiksa ketepatannya. - Jawaban soal isian memiliki kriteria penilaian yang jelas. - Tidak terdapat soal yang berulang secara tidak perlu. - Tidak ada pertemuan atau tujuan pembelajaran yang terlewat. - Seluruh soal telah dimasukkan pada kolom yang benar. - Format dokumen tetap rapi dan konsisten. H. HASIL AKHIR YANG DIMINTA Setelah seluruh proses analisis, penyusunan, pengisian, dan pemeriksaan selesai: 1. Sajikan file hasil revisi dalam format yang sama dengan file terlampir, jika memungkinkan. 2. Pastikan file dapat dibuka dan diedit kembali. 3. Jangan hanya menyajikan contoh soal atau sebagian hasil. 4. Jangan hanya memberikan kode atau instruksi untuk mengisi file secara manual. 5. Jangan meminta saya mengisi atau memperbaiki kembali bagian yang seharusnya dapat Anda selesaikan. 6. Pastikan hasil akhir benar-benar lengkap dan siap digunakan dalam pembelajaran. PRIORITAS UTAMA: KETEPATAN ISI, KESESUAIAN DENGAN TUJUAN PEMBELAJARAN, KUALITAS SOAL AKM, KETELITIAN PENGISIAN FILE, DAN KONSISTENSI JUMLAH SOAL PADA SETIAP PERTEMUAN. Lakukan pekerjaan dengan sangat teliti dan sistematis. Jangan terburu-buru menyelesaikan dokumen sebelum seluruh pertemuan dan soal selesai diperiksa.

```
Response: import file_layout
import pypdf

fields = file_layout.get_interactive_fields("ATP KELAS VII.docx")
for f in fields:
    print(f.get('name'), f.get('field_type'), f.get('rect'), f.get('options'), f.get('tooltip'))
```

User prompt: PROMPT PEMBUATAN GAME EDUKASI 3D MULTIPLAYER YANG MODERN, KEREN, DAN MENANTANG Bertindaklah sebagai Senior Game Developer, 3D Game Designer, UI/UX Designer, dan Full-Stack Developer yang berpengalaman dalam mengembangkan game edukasi 3D interaktif, multiplayer, serta berbasis sistem pembelajaran. Saya ingin membuat sebuah GAME EDUKASI 3D yang sangat menarik, modern, profesional, dan menyenangkan bagi siswa SMP. Game ini bukan sekadar permainan biasa, tetapi harus mampu memacu kemampuan berpikir, konsentrasi, kecepatan mengambil keputusan, pemahaman konsep, serta kemampuan menyelesaikan masalah sesuai dengan materi pembelajaran pada file terlampir. Game harus memiliki visual 3D yang keren, efek permainan yang memukau, tantangan yang seru, serta sistem kompetisi yang membuat siswa bersemangat untuk menyelesaikan setiap soal. JANGAN MEMBUAT GAME 2D. GAME HARUS MEMILIKI VISUAL DAN LINGKUNGAN PERMAINAN 3D YANG NYATA, INTERAKTIF, DAN MENARIK. A. KONSEP UTAMA GAME Buatlah game edukasi 3D dengan konsep permainan yang seru, menantang, dan sesuai untuk siswa SMP. Game dapat menggunakan konsep seperti: - 3D Racing Game: siswa berlomba melewati lintasan dan menjawab soal untuk meningkatkan kecepatan kendaraan. - 3D Shooting Game: siswa menembak target yang memuat pilihan jawaban yang benar. - 3D Adventure Game: siswa menjelajahi dunia 3D dan menyelesaikan tantangan pembelajaran. - 3D Space Battle: siswa menjelajahi ruang angkasa dan menyelesaikan misi edukasi. - 3D Obstacle Challenge: siswa melewati rintangan sambil menjawab soal untuk membuka jalur berikutnya. Anda boleh memilih salah satu konsep atau menggabungkan beberapa konsep tersebut. Namun, pilihlah konsep yang paling menarik, mudah dimainkan, memiliki unsur adrenalin, dan tetap berfokus pada tujuan pembelajaran. Prioritaskan game yang memiliki mekanisme sederhana tetapi memberikan pengalaman bermain yang sangat seru. Contoh konsep yang dapat dikembangkan: "INFORMATIKA FUN 3D CHALLENGE" Siswa memasuki dunia permainan 3D, menyelesaikan soal Informatika, mengumpulkan poin, membuka level baru, dan bersaing dengan pemain lain untuk menjadi juara. Nama game dapat dikembangkan menjadi nama yang lebih keren, modern, dan sesuai dengan karakter game. B. DESAIN VISUAL DAN LINGKUNGAN 3D Buatlah desain game yang memiliki kualitas visual modern dan profesional. Ketentuan desain: 1. Menggunakan lingkungan permainan 3D yang terlihat nyata dan menarik. 2. Memiliki karakter pemain, objek permainan, lintasan, arena, atau lingkungan 3D sesuai konsep game. 3. Menggunakan pencahayaan 3D yang menarik. 4. Memiliki animasi gerakan karakter atau objek yang halus. 5. Menggunakan efek visual seperti: - Particle effects. - Efek ledakan yang ramah anak. - Efek cahaya. - Efek kecepatan. - Efek pengumpulan poin. - Efek kemenangan. - Efek ketika jawaban benar atau salah. 6. Gunakan warna yang nyaman dipandang, tidak terlalu mencolok, tetapi tetap memberikan kesan energik dan futuristik. 7. Hindari desain yang terlalu ramai sehingga mengganggu konsentrasi siswa dalam membaca soal. 8. Gunakan desain UI/UX modern dengan tampilan profesional, tombol interaktif, panel informasi, dan animasi transisi. 9. Pastikan tampilan game responsif dan dapat dimainkan pada komputer, laptop, Chromebook, tablet, serta smartphone. Rekomendasi palet warna Gunakan kombinasi warna yang menarik dan nyaman: - Deep Navy: #14213D - Denim Blue: #426A9B - Electric Blue: #38BDF8 - Soft Green: #86C8A2 - Neon Mint: #6EE7B7 - White: #F8FAFC - Gold: #FBBF24 Warna dapat disesuaikan dengan lingkungan permainan, tetapi tetap pertahankan kesan modern, futuristik, dan profesional. C. SISTEM MULTIPLAYER DAN KODE BERGABUNG Game harus dapat dimainkan oleh satu orang maupun banyak orang. Sediakan sistem multiplayer dengan ketentuan berikut: 1. Pengguna dapat bermain sendiri tanpa harus menunggu pemain lain. 2. Pengguna dapat membuat ruang permainan (Room). 3. Setiap ruang permainan memiliki kode unik untuk bergabung. 4. Pengguna lain dapat bergabung menggunakan kode tersebut. 5. Siswa dapat bergabung secara mandiri. 6. Tidak ada batas jumlah pemain yang ditetapkan secara manual pada aplikasi. 7. Sistem harus dirancang agar dapat menangani banyak pemain secara bersamaan dengan baik. 8. Setiap pemain memiliki progres permainan masing-masing. 9. Pemain dapat melihat perkembangan pemain lain secara real-time. 10. Pemain yang terlambat bergabung dapat melihat status permainan dan aturan bergabung yang berlaku. 11. Sediakan fitur mulai permainan, status pemain, dan informasi ruang permainan. 12. Sediakan sistem sinkronisasi multiplayer agar data pemain tidak tertukar. Jika diperlukan, gunakan sistem room, database real-time, dan teknologi multiplayer yang sesuai. PENTING: Jangan hanya membuat tampilan kode bergabung. Kode tersebut harus benar-benar berfungsi untuk menghubungkan pemain ke ruang permainan yang sama. Sistem juga harus dirancang agar jumlah pemain dapat bertambah tanpa mengharuskan perubahan kode program setiap kali ada pemain baru. D. IDENTITAS PENGGUNA OTOMATIS Nama pengguna harus muncul secara otomatis sesuai dengan nama akun pengguna yang sedang login. Ketentuan: 1. Gunakan nama akun yang tersedia pada sistem login website. 2. Jangan meminta siswa mengetik ulang nama jika identitas akun sudah tersedia. 3. Nama pemain ditampilkan pada karakter, papan skor, progres permainan, dan hasil akhir. 4. Jika sistem memiliki akun Firebase Authentication, gunakan identitas akun tersebut sesuai struktur autentikasi yang sudah tersedia. 5. Pastikan setiap pemain memiliki identitas unik. 6. Jangan menggunakan nama akun sebagai satu-satunya dasar identifikasi data pemain. Apabila nama akun tidak tersedia, tampilkan mekanisme penanganan yang jelas tanpa menyebabkan game error. E. PROGRES PEMAIN REAL-TIME Tampilkan progres masing-masing pemain secara langsung selama permainan berlangsung. Informasi yang harus tersedia: - Nama pemain. - Level atau pertemuan yang sedang dimainkan. - Jumlah soal yang telah dijawab. - Jumlah jawaban benar. - Jumlah jawaban salah. - Skor yang diperoleh. - Persentase progres. - Waktu bermain. - Posisi pemain dalam kompetisi. - Status pemain: bermain, selesai, atau menunggu. Gunakan tampilan yang menarik, misalnya: - Progress bar. - Leaderboard real-time. - Papan skor 3D. - Animasi perubahan peringkat. - Efek ketika pemain berhasil melampaui pemain lain. - Indikator pencapaian level. Jangan membuat progres pemain hanya berupa tampilan dekoratif. Seluruh progres harus benar-benar berasal dari data permainan yang sedang berlangsung. F. INTEGRASI SOAL DARI FILE TERLAMPIR Saya melampirkan file yang berisi soal pembelajaran. Lakukan analisis terlebih dahulu terhadap seluruh isi file tersebut. Kemudian, integrasikan soal ke dalam game dengan ketentuan yang sangat ketat: 1. Gunakan soal yang terdapat dalam file terlampir. 2. Jangan mengubah kalimat soal. 3. Jangan mengubah pilihan jawaban. 4. Jangan mengubah urutan soal. 5. Jangan mengubah kunci jawaban. 6. Jangan menghilangkan soal. 7. Jangan menambahkan soal baru yang tidak terdapat dalam file. 8. Jangan memindahkan soal ke kategori yang tidak sesuai. 9. Jangan mengubah urutan kategori soal. 10. Pertahankan seluruh isi soal sebagaimana tercantum dalam file. Jika file memiliki kategori: - Pilihan Ganda. - Pilihan Ganda Kompleks. - Menjodohkan. - Isian. Maka game harus menampilkan

*soal sesuai kategori tersebut. Mekanisme permainan Buatlah mekanisme permainan yang menyesuaikan jenis soal. Contoh: Pilihan Ganda Pemain harus memilih jawaban yang benar dari pilihan yang tersedia. Jawaban yang benar memberikan poin dan memungkinkan pemain melanjutkan tantangan. Pilihan Ganda Kompleks Pemain harus memilih seluruh jawaban yang benar sesuai dengan soal. Sistem harus memeriksa kombinasi jawaban secara tepat. Menjodohkan Pemain harus memasangkan pernyataan dan jawaban yang sesuai. Gunakan mekanisme drag-and-drop atau interaksi 3D yang mudah dimainkan. Isian Pemain memasukkan jawaban pada kolom yang tersedia. Sistem harus memeriksa jawaban berdasarkan kunci dan alternatif jawaban yang telah ditentukan dalam file atau struktur data yang disediakan. Jika suatu jenis soal belum memiliki format kunci jawaban yang dapat digunakan oleh sistem, jangan mengarang kunci jawaban. Tandai bagian tersebut untuk dikonfirmasi. Urutan soal Soal harus disajikan secara berurutan sesuai file terlampir. Jangan mengacak urutan soal. Jangan membuat sistem yang secara otomatis melewati soal yang belum dijawab. Pastikan seluruh soal dapat dimainkan hingga selesai. G. SISTEM KATEGORI DAN LEVEL PERMAINAN Buatlah sistem permainan yang menarik berdasarkan kategori soal. Contoh: KategoriMekanisme GamePilihan GandaRacing ChallengePilihan Ganda KompleksTarget ShootingMenjodohkan3D Puzzle ChallengeIsianAdventure Mission Konsep tersebut dapat disesuaikan dengan desain game yang dipilih. Setiap kategori harus memiliki tantangan yang berbeda agar siswa tidak merasa bosan. Namun, pastikan mekanisme permainan tidak mengubah isi soal atau memberikan petunjuk jawaban. H. SISTEM POIN, SKOR, DAN KOMPETISI Buatlah sistem penilaian yang adil dan transparan. Ketentuan: 1. Jawaban benar mendapatkan poin. 2. Jawaban salah tidak mendapatkan poin. 3. Poin tidak boleh diberikan hanya karena pemain menekan tombol atau menyelesaikan animasi. 4. Gunakan waktu sebagai unsur tantangan tambahan, bukan satu-satunya penentu kemenangan. 5. Pemain yang memahami materi harus tetap memiliki peluang menang meskipun tidak selalu menjadi pemain tercepat. 6. Skor harus dihitung secara otomatis. 7. Skor setiap pemain harus tersimpan secara konsisten. 8. Hindari manipulasi skor melalui pengiriman data dari browser yang tidak tervalidasi. 9. Jika dua pemain memiliki skor sama, gunakan aturan penentuan peringkat yang jelas. 10. Tampilkan leaderboard secara real-time. Buatlah pengalaman kompetisi yang menyenangkan, bukan kompetisi yang membuat siswa merasa tertekan. I. KATA-KATA PENYEMANGAT BAHASA MAKASSAR-SINJAI Setiap kali siswa menyelesaikan soal atau kategori permainan, tampilkan kata-kata penyemangat secara otomatis. Gunakan bahasa yang keren, akrab, positif, dan dekat dengan siswa Makassar-Sinjai. Contoh: - "Mantap, nak! Gas terus, jangan ki' mundur!" - "Ewako! Jawabanmu mantap sekali!" - "Keren! Teruski' berjuang, tinggal sedikit lagi!" - "Baji' sekali! Otak ta' makin tajam!" - "Sipp! Lanjutki' ke tantangan berikutnya!" - "Jago sekali! Pertahankan semangat ta'!" - "Tabe', jangan menyerah! Coba lagi, pasti bisa!" - "Mantap jiwa! Selesaikan semua tantangan!" Ketentuan: 1. Kata-kata penyemangat harus muncul otomatis setelah siswa menjawab atau menyelesaikan kategori. 2. Gunakan variasi kalimat agar tidak monoton. 3. Sesuaikan kalimat dengan kondisi pemain, seperti jawaban benar, jawaban salah, progres meningkat, atau berhasil menyelesaikan level. 4. Hindari kata-kata yang merendahkan siswa. 5. Jangan mempermalukan siswa yang memperoleh skor rendah. 6. Gunakan bahasa yang positif, menyenangkan, dan sesuai dengan budaya lokal. J. SISTEM HADIAH DAN PODIUM PEMENANG Pada akhir permainan, tampilkan halaman hasil akhir yang menarik. Sediakan podium 3D untuk tiga pemain terbaik. <grid columns={3} gap={2}> <grid-item> <AsyncImage query="3D gold trophy first place winner gaming award" aspectRatio="1:1"/> **Juara 1** Piala Emas </grid-item> <grid-item> <AsyncImage query="3D silver trophy second place gaming award" aspectRatio="1:1"/> **Juara 2** Piala Perak </grid-item> <grid-item> <AsyncImage query="3D bronze trophy third place gaming award" aspectRatio="1:1"/> **Juara 3** Piala Perunggu </grid-item> </grid> Ketentuan: 1. Tampilkan tiga pemain terbaik berdasarkan skor akhir. 2. Gunakan podium 3D yang menarik. 3. Tampilkan nama pemain secara otomatis. 4. Tampilkan skor dan pencapaian masing-masing pemain. 5. Berikan animasi kemenangan. 6. Gunakan efek confetti, cahaya, dan suara kemenangan yang menyenangkan. 7. Tampilkan pesan penyemangat kepada seluruh peserta. 8. Pemain yang belum masuk tiga besar tetap mendapatkan apresiasi. 9. Jangan menetapkan pemenang hanya berdasarkan kecepatan. 10. Pastikan hasil akhir berasal dari data permainan yang telah selesai. K. KONTROL GAME YANG MUDAH Game harus mudah dimainkan oleh siswa SMP. Sediakan kontrol yang sesuai dengan perangkat. Untuk komputer atau laptop: - WASD atau tombol panah untuk bergerak. - Mouse untuk mengarahkan objek jika diperlukan. - Space untuk aksi tertentu. - Enter untuk melanjutkan. - Tombol interaktif untuk menjawab soal. Untuk smartphone atau tablet: - Joystick virtual. - Tombol aksi. - Tombol pilihan jawaban. - Kontrol sentuh yang responsif. Pastikan kontrol tidak terlalu rumit. Sediakan tutorial singkat sebelum permainan dimulai agar siswa memahami cara bermain. L. TEKNOLOGI DAN ARSITEKTUR GAME Gunakan teknologi yang sesuai untuk membangun game 3D berbasis web. Pertimbangkan: - Three.js atau Babylon.js untuk visual 3D. - JavaScript atau TypeScript. - HTML dan CSS untuk antarmuka. - Firebase Authentication untuk identitas pengguna jika sudah tersedia. - Firebase Realtime Database atau Firestore untuk sinkronisasi data. - Sistem backend yang aman untuk pengelolaan room, progres, skor, dan leaderboard. Jika website INFORMATIKA FUN sudah menggunakan Firebase, gunakan konfigurasi dan struktur yang telah tersedia. JANGAN MEMBUAT FIREBASE BARU JIKA SISTEM FIREBASE SUDAH TERSEDIA. Perhatikan: 1. Jangan mengubah sistem login yang sudah berjalan. 2. Jangan menghapus menu atau fungsi website yang telah tersedia. 3. Jangan merusak struktur website INFORMATIKA FUN. 4. Buat game sebagai modul yang terintegrasi dengan sistem yang ada. 5. Gunakan struktur file yang rapi dan mudah dipelihara. 6. Pastikan kode tidak menyebabkan konflik dengan file JavaScript lainnya. 7. Optimalkan performa agar game tetap berjalan lancar pada Chromebook dan perangkat siswa. 8. Gunakan sistem pengelolaan data yang aman dan terstruktur. M. PENGUJIAN GAME Sebelum menyatakan game selesai, lakukan pengujian secara menyeluruh. Uji hal-hal berikut: - Game dapat dibuka tanpa error. - Visual game benar-benar 3D. - Game dapat dimainkan oleh satu orang. - Game dapat dimainkan oleh banyak pemain. - Kode room dapat dibuat dan digunakan. - Pemain dapat bergabung secara mandiri. - Nama akun muncul secara otomatis. - Progres setiap pemain tampil secara real-time. - Soal sesuai dengan file terlampir. - Urutan soal tidak berubah. - Tidak ada soal yang diubah atau dihilangkan. - Sistem jawaban benar dan salah berfungsi. - Skor dihitung secara tepat. - Leaderboard berfungsi. - Kata-kata penyemangat muncul otomatis. - Podium juara 1, 2, dan 3 tampil dengan benar. - Game dapat dimainkan melalui komputer dan smartphone. - Tidak terdapat error pada console. - Tidak terdapat konflik dengan website INFORMATIKA FUN. N. HASIL AKHIR YANG SAYA INGINKAN Saya menginginkan game edukasi 3D yang benar-benar berfungsi, bukan hanya desain tampilan atau mockup. Game harus memiliki: - Visual 3D yang keren dan profesional. - Gameplay yang menyenangkan dan menantang. - Sistem multiplayer dengan kode bergabung. - Nama pemain otomatis. - Progres real-time. - Soal sesuai file terlampir. - Sistem skor dan leaderboard. - Kata-kata penyemangat bahasa Makassar-Sinjai. - Podium 3D untuk tiga pemain terbaik. - Kontrol yang mudah. - Performa yang baik pada perangkat siswa. - Integrasi yang aman dengan website pembelajaran. JANGAN HANYA MENYAJIKAN GAMBAR, DESAIN UI, ATAU CONTOH KODE YANG BELUM BERFUNGSI. Buatlah game yang dapat benar-benar dimainkan oleh siswa. Jika terdapat bagian yang belum dapat dijalankan karena membutuhkan konfigurasi Firebase, server, atau layanan tambahan, jelaskan secara spesifik bagian yang membutuhkan konfigurasi tersebut. Jangan mengklaim fitur sudah berfungsi jika belum diuji. Lakukan pengembangan secara bertahap, sistematis, dan teliti. Pastikan setiap fitur berfungsi sebelum melanjutkan ke fitur berikutnya. HASIL AKHIR HARUS MENJADI GAME EDUKASI 3D YANG MEMBUAT SISWA BERSEMANGAT BELAJAR, TERTANTANG UNTUK BERPIKIR, DAN MENIKMATI PROSES PEMBELAJARAN.*

Response: Tentu, saya akan membantu Anda membuat game edukasi 3D tersebut. Game ini akan dirancang menggunakan Three.js untuk grafis 3D dan diintegrasikan dengan Firebase untuk fitur multiplayer. Mari kita buat game "INFORMATIKA FUN 3D CHALLENGE" yang seru dan edukatif.

Penjelasan Umum Solusi

Game "INFORMATIKA FUN 3D CHALLENGE" akan mengusung konsep **3D Obstacle Challenge & Target Shooting**. Pemain akan berada di sebuah platform 3D. Di depan mereka, akan muncul soal dan beberapa target yang mewakili pilihan jawaban. Pemain harus bergerak dan "menembak" atau "menyentuh" target yang berisi jawaban yang benar untuk maju ke pertanyaan selanjutnya.

Berikut adalah komponen utama dari solusi ini:

1. **Teknologi Inti**:

- **Three.js**: Digunakan untuk me-render lingkungan 3D, karakter (sederhana), target jawaban, dan efek partikel (ledakan saat jawaban benar).
- **Firebase (Realtime Database & Auth)**: Digunakan untuk autentikasi otomatis (jika ada) dan mengelola status *room* (ruang bermain), posisi, skor, serta status jawaban pemain lain secara *real-time*.
- **HTML/CSS/JS (Vanilla)**: Mengatur UI/UX modern (menu utama, *leaderboard*, progres, dan notifikasi).

2. **Alur Permainan**:
   - **Menu Utama**: Pemain masuk, nama akun (dari Firebase Auth) otomatis terdeteksi. Pemain bisa membuat ruang (*Create Room*) atau bergabung dengan kode unik (*Join Room*).
   - **Lobby**: Menunggu pemain lain (jika *multiplayer*). Pembuat ruang bisa menekan tombol "Mulai".
   - **Gameplay 3D**: Soal (sesuai file terlampir) muncul di layar. Target jawaban muncul di dunia 3D. Pemain harus mengarahkan karakter (dengan *keyboard* atau kontrol sentuh) dan menembak target yang benar.
   - **Feedback & Progres**: Setiap jawaban memicu kata-kata penyemangat bahasa Makassar-Sinjai. Progres dan *leaderboard real-time* terlihat di UI.
   - **Akhir Game (Podium)**: Menampilkan 3 besar dengan visual sederhana.
3. **Pengelolaan Data Soal**: Soal dari file terlampir akan diubah menjadi format JSON di dalam kode agar mudah dimuat berdasarkan level/kategori.
4. **Tantangan**: Tantangannya bukan hanya kecepatan, tapi akurasi. Jawaban salah tidak memberi poin dan mungkin memberi sedikit *delay*.

Kode dan Petunjuk Penerapan

Karena proyek ini cukup besar (melibatkan 3D dan backend), saya akan memberikan kerangka kode HTML yang mencakup struktur dasar, styling (CSS) untuk UI yang modern, dan logika utama game menggunakan Three.js serta simulasi Firebase (agar Anda bisa mengujinya langsung tanpa konfigurasi awal).

Langkah 1: Persiapan File HTML

Buat file `index.html`. Kode ini menggunakan modul Three.js dari CDN.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
    <title>INFORMATIKA FUN 3D CHALLENGE</title>
    <style>
        /* CSS untuk UI yang modern dan futuristik */
        :root {
            --bg-color: #14213D;
            --primary-color: #38BDF8;
            --secondary-color: #426A9B;
            --accent-green: #86C8A2;
            --accent-mint: #6EE7B7;
            --text-light: #F8FAFC;
            --gold: #FBBF24;
        }

        body, html {
            margin: 0;
            padding: 0;
            width: 100%;
            height: 100%;
            overflow: hidden;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: var(--bg-color);
            color: var(--text-light);
        }

        /* Container 3D Canvas */
        #game-container {
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            z-index: 1; /* Di bawah UI */
        }

        /* Layar UI Overlay */
        #ui-layer {
            position: absolute;
            top: 0;
```

```
    left: 0;
    width: 100%;
    height: 100%;
    z-index: 10;
    pointer-events: none; /* Biarkan klik tembus ke canvas kecuali pada elemen UI tertentu */
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* Tampilan Menu Utama & Lobby */
.overlay-screen {
    position: absolute;
    top: 0; left: 0; width: 100%; height: 100%;
    background: rgba(20, 33, 61, 0.9);
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    pointer-events: auto; /* Tangkap klik */
    z-index: 20;
    transition: opacity 0.5s;
}

.hidden {
    opacity: 0;
    pointer-events: none !important;
}

.panel {
    background: rgba(66, 106, 155, 0.4);
    padding: 30px;
    border-radius: 15px;
    border: 2px solid var(--primary-color);
    text-align: center;
    box-shadow: 0 0 20px rgba(56, 189, 248, 0.5);
    max-width: 400px;
    width: 90%;
}

h1 { color: var(--gold); text-shadow: 0 0 10px var(--gold); margin-bottom: 20px;}
h2 { color: var(--primary-color); }

input {
    width: 80%;
    padding: 10px;
    margin: 10px 0;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    text-align: center;
    background: rgba(255,255,255,0.9);
}

button {
    background: linear-gradient(45deg, var(--primary-color), var(--secondary-color));
    color: white;
    border: none;
    padding: 12px 25px;
    margin: 10px;
    border-radius: 8px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
    transition: transform 0.2s, box-shadow 0.2s;
    text-transform: uppercase;
}
button:hover {
    transform: scale(1.05);
    box-shadow: 0 0 15px var(--primary-color);
}

/* UI Saat Bermain */
#hud {
    width: 100%;
    padding: 20px;
    box-sizing: border-box;
    display: flex;
    justify-content: space-between;
```

```
            pointer-events: auto;
        }

        .hud-panel {
            background: rgba(20, 33, 61, 0.7);
            border: 1px solid var(--primary-color);
            border-radius: 10px;
            padding: 15px;
        }

        #question-panel {
            flex-grow: 1;
            margin: 0 20px;
            text-align: center;
            font-size: 18px;
            font-weight: bold;
            background: rgba(66, 106, 155, 0.8);
            border: 2px solid var(--accent-mint);
        }

        #leaderboard {
            width: 250px;
        }
        .lb-entry { display: flex; justify-content: space-between; margin: 5px 0; border-bottom: 1px solid

        /* Umpan Balik (Makassar-Sinjai) */
        #feedback-msg {
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            font-size: 32px;
            font-weight: bold;
            color: var(--gold);
            text-shadow: 0 0 20px var(--gold);
            opacity: 0;
            transition: opacity 0.3s;
            z-index: 15;
            pointer-events: none;
        }

        /* Kontrol Mobile (Disembunyikan di Desktop) */
        #mobile-controls {
            position: absolute;
            bottom: 30px;
            width: 100%;
            display: flex;
            justify-content: space-around;
            pointer-events: auto;
        }
        .ctrl-btn {
            width: 70px; height: 70px;
            border-radius: 50%;
            background: rgba(56, 189, 248, 0.5);
            border: 2px solid var(--primary-color);
            color: white; font-size: 24px; font-weight: bold;
            display: flex; justify-content: center; align-items: center;
            user-select: none;
        }
        @media (min-width: 768px) { #mobile-controls { display: none; } }
    </style>
</head>
<body>

    <!-- Container 3D -->
    <div id="game-container"></div>

    <!-- UI Overlay -->
    <div id="ui-layer">
        <!-- HUD Saat Bermain -->
        <div id="hud" class="hidden">
            <div class="hud-panel" id="player-info">
                Pemain: <span id="p-name">Player1</span><br>
                Skor: <span id="p-score">0</span>
            </div>
            <div class="hud-panel" id="question-panel">
                Memuat soal...
            </div>
            <div class="hud-panel" id="leaderboard">
```

```
            <h3 style="margin-top:0">Leaderboard</h3>
            <div id="lb-list">
                <!-- List pemain muncul di sini -->
            </div>
        </div>
    </div>

    <div id="feedback-msg">Ewako!</div>

    <!-- Kontrol Sentuh -->
    <div id="mobile-controls" class="hidden">
        <div class="ctrl-btn" id="btn-left">←</div>
        <div class="ctrl-btn" id="btn-shoot">Tembak</div>
        <div class="ctrl-btn" id="btn-right">→</div>
    </div>
</div>

<!-- Menu Utama -->
<div id="menu-screen" class="overlay-screen">
    <div class="panel">
        <h1>INFORMATIKA FUN<br>3D CHALLENGE</h1>
        <p>Identitas: <strong id="auth-name">Mengambil data akun...</strong></p>
        <input type="text" id="room-code-input" placeholder="Masukkan Kode Room (Maks 5 huruf)" maxleng
        <br>
        <button id="btn-join">Gabung Room</button>
        <button id="btn-create">Buat Room Baru</button>
    </div>
</div>

<!-- Lobby Menunggu -->
<div id="lobby-screen" class="overlay-screen hidden">
    <div class="panel">
        <h2>Ruang Tunggu</h2>
        <p>Kode Room: <strong id="lobby-code" style="font-size: 24px; color: var(--gold);">---</strong>
        <p>Pemain Terhubung:</p>
        <ul id="lobby-players" style="list-style: none; padding: 0; max-height: 150px; overflow-y: auto
            <!-- Daftar pemain di lobby -->
        </ul>
        <button id="btn-start" class="hidden">Mulai Game</button>
        <p id="waiting-msg">Menunggu Host memulai...</p>
    </div>
</div>

<!-- Podium Akhir -->
<div id="podium-screen" class="overlay-screen hidden">
     <div class="panel" style="max-width: 600px;">
        <h1>HASIL AKHIR</h1>
        <h2 style="color:var(--accent-mint)">Mantap Jiwa! Selesai Mi Semua Tantangan!</h2>
        <div style="display: flex; justify-content: space-around; margin-top: 30px; align-items: flex-e
            <div style="text-align: center;">
                <div style="font-size: 30px;">🥈</div>
                <div style="background: silver; width: 60px; height: 80px; margin: 0 auto; display:flex
                <div id="pos-2-name">---</div>
            </div>
            <div style="text-align: center;">
                <div style="font-size: 40px;">🏆</div>
                <div style="background: gold; width: 60px; height: 120px; margin: 0 auto; display:flex
                <div id="pos-1-name">---</div>
            </div>
            <div style="text-align: center;">
                <div style="font-size: 30px;">🥉</div>
                <div style="background: #cd7f32; width: 60px; height: 50px; margin: 0 auto; display:fle
                <div id="pos-3-name">---</div>
            </div>
        </div>
        <button id="btn-home" style="margin-top: 30px;">Kembali ke Menu</button>
    </div>
</div>

<!-- Library Three.js -->
<script type="importmap">
    {
        "imports": {
            "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
            "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
        }
    }
</script>
```

```
<!-- Script Logika Game -->
<script type="module">
    import * as THREE from 'three';

    // --- DATA SOAL (Diambil dari File Terlampir - Bab 1 PG sebagai contoh) ---
    const questionsData = [
        {
            q: "Saat Budi pergi ke minimarket, ia menerima struk belanja yang berisi nama barang, harga
            options: [
                "Tercetak di atas kertas thermal",
                "Memiliki format, kolom, dan baris yang jelas",
                "Dapat dibaca oleh semua orang",
                "Dihasilkan oleh mesin kasir"
            ],
            correct: 1 // Index jawaban benar (B)
        },
        {
            q: "Manakah dari data berikut yang paling sulit dianalisis secara langsung oleh komputer ta
            options: [
                "Daftar hadir siswa format Excel",
                "Rekaman suara wawancara",
                "Tabel nilai ujian tengah semester",
                "Jadwal kereta api di website resmi"
            ],
            correct: 1
        },
        {
            q: "Siti merapikan kamarnya dan menemukan tumpukan foto liburan, buku harian, dan tabel jad
            options: [
                "Tabel jadwal pelajaran",
                "Foto liburan dan buku harian",
                "Tabel jadwal dan buku harian",
                "Hanya foto liburan"
            ],
            correct: 1
        }
    ];

    // --- KATA PENYEMANGAT ---
    const cheers = [
        "Mantap, nak! Gas terus!",
        "Ewako! Jawabanmu mantap sekali!",
        "Baji' sekali! Otak ta' makin tajam!",
        "Jago sekali! Pertahankan semangat ta'!"
    ];

    // --- VARIABEL GLOBAL ---
    let currentQuestionIndex = 0;
    let myScore = 0;
    let myName = "Siswa_" + Math.floor(Math.random() * 1000); // Dummy jika auth belum siap
    let currentRoom = "";
    let isHost = false;

    // Objek 3D
    let scene, camera, renderer;
    let playerShip;
    let targetMeshes = [];
    let projectiles = [];
    let particles = [];

    // State
    let isPlaying = false;
    let keys = {};

    // Simulasi Multiplayer (Gunakan Firebase Realtime DB di tahap produksi)
    let simulatedPlayers = {};

    // --- INISIALISASI UI ---
    document.getElementById('auth-name').innerText = myName;

    document.getElementById('btn-create').addEventListener('click', () => {
        currentRoom = Math.random().toString(36).substring(2, 7).toUpperCase();
        isHost = true;
        enterLobby(currentRoom);
    });

    document.getElementById('btn-join').addEventListener('click', () => {
        const code = document.getElementById('room-code-input').value.toUpperCase();
        if(code.length === 5) {
```

```
        currentRoom = code;
        isHost = false;
        enterLobby(currentRoom);
    } else {
        alert("Masukkan 5 huruf kode room!");
    }
});

function enterLobby(code) {
    document.getElementById('menu-screen').classList.add('hidden');
    document.getElementById('lobby-screen').classList.remove('hidden');
    document.getElementById('lobby-code').innerText = code;

    updateLobbyList();

    if(isHost) {
        document.getElementById('btn-start').classList.remove('hidden');
        document.getElementById('waiting-msg').classList.add('hidden');
    }
}

document.getElementById('btn-start').addEventListener('click', startGame);
document.getElementById('btn-home').addEventListener('click', () => location.reload());

function updateLobbyList() {
    // Simulasi: Masukkan diri sendiri dan 1 bot
    const ul = document.getElementById('lobby-players');
    ul.innerHTML = `<li>🎮 ${myName} (Anda)</li><li>🤖 Daeng Bot (Menunggu)</li>`;

    simulatedPlayers[myName] = { score: 0, q: 0 };
    simulatedPlayers["Daeng Bot"] = { score: 0, q: 0 };
}

// --- INISIALISASI 3D (Three.js) ---
function init3D() {
    const container = document.getElementById('game-container');

    scene = new THREE.Scene();
    scene.background = new THREE.Color(0x14213D); // Deep Navy
    scene.fog = new THREE.FogExp2(0x14213D, 0.02);

    camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 1000);
    camera.position.set(0, 5, 15);
    camera.lookAt(0, 0, 0);

    renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    container.appendChild(renderer.domElement);

    // Pencahayaan
    const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
    scene.add(ambientLight);

    const dirLight = new THREE.DirectionalLight(0x38BDF8, 0.8);
    dirLight.position.set(10, 20, 10);
    scene.add(dirLight);

    // Lantai Grid (Gaya Futuristik)
    const gridHelper = new THREE.GridHelper(100, 50, 0x38BDF8, 0x426A9B);
    gridHelper.position.y = -2;
    scene.add(gridHelper);

    // Karakter Pemain (Pesawat Sederhana)
    const geometry = new THREE.ConeGeometry(1, 2, 4);
    const material = new THREE.MeshPhongMaterial({ color: 0xFBBF24, flatShading: true });
    playerShip = new THREE.Mesh(geometry, material);
    playerShip.rotation.x = Math.PI / 2;
    scene.add(playerShip);

    window.addEventListener('resize', onWindowResize, false);

    // Input Keyboard
    window.addEventListener('keydown', (e) => keys[e.code] = true);
    window.addEventListener('keyup', (e) => keys[e.code] = false);
    // Tombol Spasi untuk menembak
    window.addEventListener('keydown', (e) => { if(e.code === 'Space' && isPlaying) shoot(); });

    // Input Touch Mobile
    document.getElementById('btn-left').addEventListener('touchstart', (e)=>{e.preventDefault(); ke
```

```
    document.getElementById('btn-left').addEventListener('touchend', (e)=>{e.preventDefault(); keys
    document.getElementById('btn-right').addEventListener('touchstart', (e)=>{e.preventDefault(); k
    document.getElementById('btn-right').addEventListener('touchend', (e)=>{e.preventDefault(); key
    document.getElementById('btn-shoot').addEventListener('touchstart', (e)=>{e.preventDefault(); s

    animate();
}

function onWindowResize() {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
}

// --- LOGIKA GAME ---
function startGame() {
    document.getElementById('lobby-screen').classList.add('hidden');
    document.getElementById('hud').classList.remove('hidden');
    document.getElementById('p-name').innerText = myName;

    // Tampilkan kontrol mobile jika di perangkat kecil
    if(window.innerWidth < 768) {
        document.getElementById('mobile-controls').classList.remove('hidden');
    }

    init3D();
    isPlaying = true;
    currentQuestionIndex = 0;
    myScore = 0;
    loadQuestion();
}

function loadQuestion() {
    if (currentQuestionIndex >= questionsData.length) {
        endGame();
        return;
    }

    const qData = questionsData[currentQuestionIndex];
    document.getElementById('question-panel').innerHTML = `Pertanyaan ${currentQuestionIndex+1}/${(

    // Bersihkan target sebelumnya
    targetMeshes.forEach(m => scene.remove(m));
    targetMeshes = [];

    // Buat Target 3D (Kotak)
    const positions = [-6, -2, 2, 6]; // Posisi X untuk 4 jawaban
    const colors = [0xff4444, 0x44ff44, 0x4444ff, 0xffff44];

    for(let i=0; i<4; i++) {
        const geo = new THREE.BoxGeometry(2, 2, 2);
        const mat = new THREE.MeshPhongMaterial({ color: colors[i] });
        const mesh = new THREE.Mesh(geo, mat);
        mesh.position.set(positions[i], 1, -20);
        mesh.userData = {
            index: i,
            isCorrect: (i === qData.correct)
        };
        scene.add(mesh);
        targetMeshes.push(mesh);

        // Tambahkan teks jawaban di atas kotak (menggunakan HTML overlay untuk kemudahan render)
        // Pada implementasi penuh, gunakan TextGeometry atau CSS2DRenderer
    }

    // Update Teks Pilihan di UI agar lebih mudah dibaca
    let optionsHTML = `<div style="display:flex; justify-content:space-around; font-size:12px; marg
    qData.options.forEach((opt, idx) => {
        optionsHTML += `<div style="width:20%; background:rgba(0,0,0,0.5); padding:5px; border-radi
    });
    optionsHTML += `</div>`;
    document.getElementById('question-panel').innerHTML += optionsHTML;
}

function shoot() {
    // Buat proyektil laser
    const geo = new THREE.CylinderGeometry(0.1, 0.1, 1);
    const mat = new THREE.MeshBasicMaterial({ color: 0x6EE7B7 });
    const laser = new THREE.Mesh(geo, mat);
```

```
    laser.rotation.x = Math.PI / 2;
    laser.position.copy(playerShip.position);
    laser.position.z -= 1; // Mulai di depan kapal
    scene.add(laser);
    projectiles.push(laser);
}

function createExplosion(position, color) {
    for(let i=0; i<20; i++) {
        const geo = new THREE.BoxGeometry(0.2, 0.2, 0.2);
        const mat = new THREE.MeshBasicMaterial({color: color});
        const particle = new THREE.Mesh(geo, mat);
        particle.position.copy(position);

        // Kecepatan acak
        particle.userData = {
            velocity: new THREE.Vector3(
                (Math.random() - 0.5) * 0.5,
                (Math.random() - 0.5) * 0.5,
                (Math.random() - 0.5) * 0.5
            ),
            life: 1.0
        };
        scene.add(particle);
        particles.push(particle);
    }
}

function showFeedback(msg) {
    const el = document.getElementById('feedback-msg');
    el.innerText = msg;
    el.style.opacity = 1;
    setTimeout(() => el.style.opacity = 0, 2000);
}

function checkCollisions() {
    for(let i = projectiles.length - 1; i >= 0; i--) {
        const p = projectiles[i];
        const pBox = new THREE.Box3().setFromObject(p);

        for(let j = targetMeshes.length - 1; j >= 0; j--) {
            const t = targetMeshes[j];
            const tBox = new THREE.Box3().setFromObject(t);

            if(pBox.intersectsBox(tBox)) {
                // Kena Target!
                scene.remove(p);
                projectiles.splice(i, 1);

                scene.remove(t);
                targetMeshes.splice(j, 1);

                if(t.userData.isCorrect) {
                    createExplosion(t.position, 0xFBBF24); // Ledakan Emas
                    myScore += 10;
                    document.getElementById('p-score').innerText = myScore;

                    // Kata penyemangat acak
                    showFeedback(cheers[Math.floor(Math.random() * cheers.length)]);

                    // Lanjut soal
                    currentQuestionIndex++;
                    simulatedPlayers[myName].score = myScore;

                    // Simulasi bot ikut menjawab
                    simulatedPlayers["Daeng Bot"].score += Math.random() > 0.5 ? 10 : 0;

                    updateLeaderboardUI();
                    setTimeout(loadQuestion, 1500); // Jeda sebelum soal berikutnya
                } else {
                    createExplosion(t.position, 0xff0000); // Ledakan Merah
                    showFeedback("Tabe', kurang tepat! Coba lagi!");
                    // Tidak lanjut soal, biarkan pemain menembak target lain
                }
                break; // Keluar dari loop target
            }
        }
    }
}
```

```
        function updateLeaderboardUI() {
            // Urutkan berdasarkan skor
            const sortedPlayers = Object.keys(simulatedPlayers).sort((a, b) => simulatedPlayers[b].score -

            let html = '';
            sortedPlayers.forEach(name => {
                html += `<div class="lb-entry"><span>${name}</span><span>${simulatedPlayers[name].score} p
            });
            document.getElementById('lb-list').innerHTML = html;
        }

        function endGame() {
            isPlaying = false;
            document.getElementById('hud').classList.add('hidden');
            document.getElementById('mobile-controls').classList.add('hidden');

            const endScreen = document.getElementById('podium-screen');
            endScreen.classList.remove('hidden');

            const sortedPlayers = Object.keys(simulatedPlayers).sort((a, b) => simulatedPlayers[b].score -

            document.getElementById('pos-1-name').innerText = sortedPlayers[0] || "---";
            document.getElementById('pos-2-name').innerText = sortedPlayers[1] || "---";
            // document.getElementById('pos-3-name').innerText = sortedPlayers[2] || "---"; // Jika ada per
        }

        function animate() {
            requestAnimationFrame(animate);

            if(!isPlaying) return;

            // Pergerakan Pemain
            const speed = 0.2;
            if (keys['ArrowLeft'] || keys['KeyA']) playerShip.position.x -= speed;
            if (keys['ArrowRight'] || keys['KeyD']) playerShip.position.x += speed;

            // Batasi area gerak
            playerShip.position.x = Math.max(-8, Math.min(8, playerShip.position.x));

            // Rotasi Target (Efek melayang)
            targetMeshes.forEach(m => {
                m.rotation.y += 0.01;
                m.rotation.x += 0.005;
            });

            // Pergerakan Proyektil
            for(let i = projectiles.length - 1; i >= 0; i--) {
                projectiles[i].position.z -= 0.5;
                if(projectiles[i].position.z < -30) {
                    scene.remove(projectiles[i]);
                    projectiles.splice(i, 1);
                }
            }

            // Animasi Partikel Ledakan
            for(let i = particles.length - 1; i >= 0; i--) {
                let p = particles[i];
                p.position.add(p.userData.velocity);
                p.material.opacity = p.userData.life;
                p.material.transparent = true;
                p.userData.life -= 0.05;

                if(p.userData.life <= 0) {
                    scene.remove(p);
                    particles.splice(i, 1);
                }
            }

            checkCollisions();
            renderer.render(scene, camera);
        }

    </script>
</body>
</html>
```